

A Study on Disseminating Dependent Information in Wireless Broadcast Environments

Kun-Feng Lin and Chuan-Ming Liu

Department of Computer Science and Information
Engineering National Taipei University of Technology
Taipei 106, TAIWAN
{cmliu, ivan}@mcse.csie.ntut.edu.tw

Abstract

Data broadcasting is an effective approach to disseminate public information to mobile clients in wireless mobile environments. In some applications, the access pattern of all the data can be represented by a weighted DAG. In this paper, we explore how the server generates the broadcast schedule, for the data set having a DAG access pattern, which not only minimizes the access latency but also is a topological ordering of the DAG. We prove that it is NP-hard to find an optimal broadcast schedule and provide two heuristics. After giving an analysis for these two heuristics on the latency and complexity, we implement the proposed heuristics to compare their performance.

1 Introduction

Data broadcasting provides an efficient way to disseminate data in the wireless environments and has attracted much research attention in recent years [1, 6, 7, 10, 12, 15, 13]. In a wireless environment, a server can provide public information (e.g. electronic news service, traffic information, stock-price information, etc.) to a large client population using broadcasting. Clients can access the information by tuning into the broadcast. One key issue related to data broadcasting is how the server organizes the data in the broadcast in order that the clients can access the information efficiently in terms of the *latency* (time elapsed from requesting to receiving data) and the *tuning time* (the amount of time spent listening to the broadcast). Short latency and tuning time can ensure the quality of service (QoS) provided by the server.

Most of the previous work about the broadcast scheduling assumed that the broadcast data requested are independent [9, 17]. However, in some

applications, the data requested are dependent. For example, consider the scenario that a user requests a Web page which contains some components, such as audio clips or images. When a mobile client uses a browser to access the Web page, the browser will request all the components contained in the Web page after receiving the Web page. Such a request leads to an access pattern among the dependent data [2, 8, 10]. The other application can be found when a mobile client has an inquiry about the stock price of some company. It is highly possible that the mobile client would like to know the stock information of some other related companies. In this case, the information among all the related companies is dependently accessed. Besides, consider the on-demand broadcasting model in [16] where the server collects a batch of requests from the mobile clients and then sends the results to that group of clients via broadcast. After receiving a signal from the server, each mobile client starts to receive the result from the beginning of the broadcast cycle. The data broadcast in this model are correlated and dependent.

In order to minimize the latency, many of the previous papers considered the given data access frequency [1, 7, 9, 13, 17] or the data access pattern [2, 8, 10]. With such assumptions, how the server schedules the data is the major challenge. An important issue when designing the broadcast schedule with minimized latency is to consider whether the broadcast allows a mobile client to receive the data in the middle of a broadcast cycle. Allowing the users to receive data in the middle of a cycle can reduce the latency by skipping the waiting time for the beginning of a cycle. However, for the scenarios mentioned in the previous paragraph, it is not proper to allow a client to receive the data in the middle of a cycle due to the data dependency. We thus discuss the case that the mobile client can only start to

receive data from the beginning of a broadcast cycle. Such a consideration distinguishes our work from the others [2, 8, 10]. In this paper, we consider the access pattern formed by all the requests where the data dependency and access frequency can be represented by a weighted *directed acyclic graph*, *DAG* [10]. We investigate how to generate the broadcast schedule in order to minimize the average latency when the broadcast data has the access pattern as a weighted DAG.

According to the data dependency, we consider the generated broadcast schedule is a *topological ordering* of the vertices in G . A *topological ordering* of a DAG G is an ordering of the vertices in G where the precedent vertices in the DAG are broadcast earlier than the following vertices. In other words, for an edge (u, v) in a DAG G , vertex u should be broadcast before vertex v in the same cycle. The broadcast schedule generated without considering the weights (access frequencies) and without the topological ordering may result in a longer latency. We show that it is NP-hard to find an optimal broadcast schedule which is a topological ordering of the input DAG and results in a minimum latency.

The organization for the rest of this paper is as follows. In Section 2, we give some related works as well as the terminology and notations used in this paper. We then formulate the problem formally in Section 3. After showing that the proposed problem is NP-complete in Section 4, we propose two heuristic methods. The heuristic proposed in Section 5 considers the level of each vertex and then assigns a position to each vertex. The other heuristic generates the broadcast schedule by traversing the input DAG in a greedy fashion and is discussed in Section 6. In Section 7, we show the simulation for these two heuristics. Section 8 concludes this paper.

2 Preliminary

One objective of data broadcast scheduling is to minimize the access latency. As mentioned, allowing a mobile client to start receiving the data in the middle of a broadcast cycle can reduce the access latency. Thus, many papers considered to minimize the relative distance between the positions of two related data items in order to minimize the access latency [2, 4, 10, 13]. In [2], the authors considered three types of relation between two accessed data items, including *AND*, *OR*, and *IMPLY* relation. They assumed that it takes one

unit of time (a *slot*) to broadcast each file. For each kind of relation, they provided a heuristic to generate the broadcast which minimizes the latency. In [13], the authors considered data allocation and query processing on wireless broadcast channels. They assumed that the access pattern can be modeled as a *directed graph*. In their solution, the directed graph was transferred into an access forest by cutting some edges. Each tree in the forest may have an optimal schedule locally. Then the broadcast schedule was generated by combining those optimal schedules of the trees in a global viewpoint. However, the resulting schedule may not be optimal in terms of latency since some edges had been removed when transferring the graph into a forest. Besides, [14] considered a graph having the distance cost between any two vertices as well as the vertex weighted cost. The objective is to find the minimum back-walk-free latency for such given graph. However the optimal solution of the minimum back-walk-free latency problem does not have the linear ordering.

[4] assumed that the relation among the data to be accessed can be represented by a *directed acyclic graph* (DAG) and presented several heuristics according to the following three criteria, (1) linear ordering (*i.e.* topological ordering), (2) minimum linear distance between any two related objects and (3) more availability for popular objects. The authors considered either the case that the generated schedule is a topological ordering for an un-weighted DAG or the case that the resulting schedule is not necessarily a topological ordering for a weighted DAG. In our work, the generated broadcast schedule for a weighted DAG is a topological ordering. We consider the case that each mobile client starts to receive data from the beginning of a broadcast cycle and focus on how to minimize the latency in an overall viewpoint. In the next section, we will give our problem a formal definition.

A DAG [3] $G = (V, E)$ is a directed graph that has no cycles, where V is a finite set of vertices and E is a binary relation on V . The set E is called the *edge set* of G and consists of ordered pairs of vertices. An element (u, v) in E , $u, v \in V$ and $u \neq v$, is an *edge*. If (u, v) is an edge in G , we say that vertex u is *adjacent* to (from) vertex v . We also call the edge (u, v) as the *in-edge* (*out-edge*) of v and the *in-degree* (*out-degree*) of a vertex is the number of edges entering (leaving) it. We define *in-degree* (*out-degree*) $d_i(d_o)$ of a DAG G to be the maximum in-degree (out-degree) among all the vertices in G . The *degree* d of a

DAG G is $\max\{d_i, d_o\}$. Among the vertices in V , a *source* is a vertex which has no in-edge and a *sink* is a vertex having no out-edge. If there is a path $P(u, v)$ from u to v , we say that v is *reachable* from u via $P(u, v)$ and u is a prior vertex of v as well as v is a posterior vertex of u . For a vertex v , we further define $ADJ(v)$ as the set of all the vertices adjacent from v .

For each edge (u, v) in the input DAG, we consider that there is a weight $p_{uv} < 1$ on that edge. For any edge (u, v) , the direction from vertex u to vertex v represents that u is accessed before v and the weight $p_{uv} \leq 1$ on (u, v) stands for the conditional probability of accessing vertices v after vertex u has been accessed. As shown in Figure 1, after vertex a has been accessed, the chances to access vertex b , c , and d are $p_{ab} = 0.8$, $p_{ac} = 0.01$, and $p_{ad} = 0.1$, respectively. The summation of all the weights on the edges from a vertex u is less than or equal to 1.

3 Problems

Suppose that the access pattern of the data to be broadcasted is modeled as an edge-weighted DAG $G = (V, E)$. We assume that there is only one source s in the DAG. If the input DAG has multiple sources, we add a pseudo-vertex s to the DAG which has a directed edge to each source with equal weight respectively.

For each vertex v except the source s in the DAG G , there is at least one path from s to vertex v . Suppose $P(s, v)$ is one of the paths from s to v . We define the probability to access vertex v via path $P(s, v)$ as $\prod_{(i,j) \in P(s,v)} p_{ij}$. Let $\mathbb{P}_v$ be the set of all the paths from the source s to vertex v . The probability to access vertex v from the source s is

$$\gamma(v) = \sum_{P(s,v) \in \mathbb{P}_v} \left(\prod_{(i,j) \in P(s,v)} p_{ij} \right). \quad (1)$$

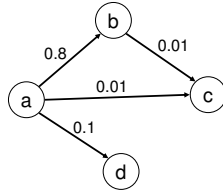


Figure 1: A weighted DAG of 4 vertices; the weight on an edge (u, v) representing the access probability for vertex v after vertex u has been accessed.

For example, consider the DAG in Figure 1. The $\gamma(c) = 0.8 \times 0.01 + 0.01 = 0.018$.

Suppose that a generated broadcast schedule f is a 1-to-1 mapping from V to the set $\{1, \dots, |V|\}$. Then the position of vertex v in the broadcasting channel is $f(v)$. The expected latency for accessing vertex v via the broadcast schedule f is therefore $f(v) \cdot \gamma(v)$. Hence the *overall latency* for accessing all the data can be

$$\sum_{v \in V} f(v) \cdot \gamma(v). \quad (2)$$

This paper discusses the problem about how to generate a broadcast schedule which (1) is a topological ordering of the vertices in the input DAG and (2) has the minimum overall latency as in Equation 2. We refer to such a problem as the *Topological Ordering Broadcast with Minimum Latency (TOBML) Problem* and give a formal definition for it in Definition 1. In the following sections, we will show that the TOBML problem is NP-complete and provide some heuristics for it.

Definition 1 (TOBML)

Suppose all the notations are defined as above. The *Topological Ordering Broadcast with Minimum Latency Problem* is to find a 1-to-1 mapping $f : V \rightarrow \{1, 2, \dots, |V|\}$ such that

1. if $(u, v) \in E$, then $f(u) < f(v)$, and
2. $\sum_{v \in V} f(v) \cdot \gamma(v)$ is minimized. □

4 NP-completeness

To show that the TOBML problem is NP-Complete, we consider the decision problem, *Topological Ordering Broadcast with a Given Latency (TOBGL) Problem*, defined as below.

Definition 2 (TOBGL)

Instance: A directed acyclic graph (DAG) $G = (V, E)$, an integer K , and a weight $p_{uv} \leq 1$ for each edge $(u, v) \in E$.

Question: Does there exist a 1-to-1 mapping $f : V \rightarrow \{1, 2, \dots, |V|\}$ such that

1. if $(u, v) \in E$, then $f(u) < f(v)$, and
2. $\sum_{v \in V} f(v) \cdot \gamma(v) \leq K$. □

Clearly, if the TOBGL problem is NP-complete, then TOBML is also NP-complete. We therefore focus on showing TOBGL is NP-complete. Consider a special type of DAG defined in Definition 3 for the TOBGL problem.

Definition 3 (*Equally Out-edge Weight DAG*)

An Equally Out-edge Weight DAG is a DAG in which (1) all the edges from a vertex have the same weight and (2) the sum of all the weights on the edges from a vertex is equal to 1. $\square$

When the input is an equally out-edge weight DAG for the TOBGL problem, we refer to such a problem as the ETOBGL problem. We then claim that the ETOBGL problem is NP-complete. ETOBGL is in NP since, given a 1-to-1 mapping f , it takes linear time to determine whether $f(u) < f(v)$ if $(u, v) \in E$ and whether $\sum_{v \in V} f(v) \cdot \gamma(v) \leq K$. To show that ETOBGL is NP-hard, we present a reduction from the *Minimum Linear Arrangement of Directed Graph (MINLARD) Problem* defined in Definition 4 which is NP-hard [5].

Definition 4 (*MINLARD*)

Instance: A directed acyclic graph (DAG) $G' = (V', E')$ and an integer K' .

Question: Does there exist a 1-to-1 mapping $g : V' \rightarrow \{1, 2, \dots, |V'|\}$ such that

1. if $(u, v) \in E'$, then $g(u) < g(v)$, and
2. $\sum_{(u,v) \in E'} [g(v) - g(u)] \leq K'$. $\square$

In the following, we show the reduction. Suppose I' is an instance of the MINLARD problem. An instance I of the ETOBGL problem can be constructed from I' as follows.

1. Let $V = V'$ and $E = E'$.
2. For each v in V , all the out-edges of v have the same weight, $\frac{1}{d_v}$ which is the inverse of v 's out-degree d_v .
3. The value of K can be derived by the following relation between K and K' . Suppose the 1-to-1 mapping g exists in I' . We then consider $K = \sum_{v \in V} \gamma(v) f(v)$. Note that g is a permutation of f and for each vertex v in V , there exists a vertex u in V such that $f(v) = g(u)$. Then we can get

$$\begin{aligned} K &= \sum_{v \in V} \gamma(v) f(v) = \sum_{v \in V'} \gamma(v) g(u) \\ &= \sum_{v \in V'} \gamma(v) \cdot \{[g(u) - g(a_1)] + [g(a_1) - g(a_2)] + \dots + [g(a_l) - g(s)] + g(s)\}, \end{aligned}$$

where $a_1, a_2, \dots, a_l$ are the vertices along the longest path from s to u . Since $g(s) = 1$,

$$K \leq \sum_{v \in V'} \gamma(v) \cdot \left\{ \sum_{(i,j) \in E'} g(j) - g(i) + 1 \right\}$$

By the property of the 1-to-1 mapping g ,

$$K \leq \sum_{v \in V'} \gamma(v) \cdot \{K' + 1\} = \{K' + 1\} \sum_{v \in V'} \gamma(v),$$

where $\sum_{v \in V'} \gamma(v)$ is a known constant.

Then, given an arbitrary instance I' of the MINLARD problem, we can construct an instance I of the ETOBGL problem and such a construction can be done in $O(m)$ time where m is the number of edges in the input DAG. It then remains to show that there is a solution for an instance I' of the MINLARD problem if and only if there is a solution for the instance I of the ETOBGL problem. However, this can be done easily by following the relation between K and K' . Hence, we can have the following conclusion:

Theorem 1 *The ETOBGL problem is NP-complete; therefore, the TOBML problem is NP-complete.* $\square$

5 Level-Oriented Heuristic

In the previous section, we have shown that the TOMBL problem is NP-complete. In this and the following sections, we introduce two heuristics for generating the broadcast schedules. One heuristic first considers the level of each vertex and then assigns the position to each vertex. The other heuristic generates the broadcast schedule by traversing the input DAG in a *greedy* fashion and will be discussed in Section 6.

5.1 Forward Minimum Offset Algorithm

This subsection introduce the *Forward Minimum Offset Algorithm (FMOA)*. For a vertex v , we define the *level* of v is the number of vertices on the longest path from the source to v itself. The basic idea of FMOA is as follows. Suppose f^* and f are the best broadcast schedule and a generated schedule, respectively, and each of both schedules is a topological ordering of the vertices. The objective is to minimize $\sum_{v \in V} |f(v) - f^*(v)|$. To achieve this, we first consider the level of each vertex instead of the exact position of each vertex in the broadcast. Note that the levels of the vertices confine the placement for the vertices and implicitly present the topological ordering among the vertices. Furthermore, the level of a vertex v is the best possible position for v to be placed according to the topological ordering. A vertex

v having a smaller level should be broadcast earlier. After having the level of each vertex, we then assign each vertex a position in the broadcast.

FMOA therefore has two major steps: (1) *Assigning Level* step and (2) *Positioning with Minimum Offset* step. Assigning Level step assigns each vertex a level. The source s must be on level 1 according to the topological ordering of the vertices. All the vertices adjacent from s therefore should be on level 2. Process continues based on the following rule: *suppose vertex u is adjacent to vertex v , the level of u must be smaller than the level of v* . When every vertex has been assigned to a level, we move on to the next major step. Positioning with Minimum Offset step places a vertex at a position in the broadcast based on the level of a vertex. A vertex having a smaller level will be placed at an earlier position. If two or more vertices have the same level, we use the γ value to decide the position. A vertex with larger γ value will be placed at an earlier position. After these two steps, we then have a broadcast schedule which minimizes the latency and is a topological ordering of the vertices. Figure 2 shows the high-level description of the FMOA algorithm.

We now use the DAG in Figure 3 to illustrate how FMOA generates the broadcast schedule. In the Assigning Level step, the level of vertex a is 1. Vertices b, c, d, e and g are adjacent from a and form the set $ADJ(a)$. Then, the algorithm assigns level 2 to vertices b, d, c, e and g . After all the vertices in $ADJ(a)$ have been assigned, the algorithm then continues to traverse the vertices in $ADJ(a)$. By removing vertex a and all the edges incident from a , we can have new sources c and d in $ADJ(a)$.

For simplicity and convenience, we store the sources into a queue Q . The algorithm then can traverse vertex c or d . Suppose vertex d is extracted to be traversed next. When vertex d is traversed, since vertex d is assigned to level 2, the vertices b, e , and f adjacent from d are assigned to level 3. Consider vertices b and e which has been assigned to level 2 before. To keep the topological ordering, vertices b and e now should be reassigned to level 3. After removing vertex d and all the edges incident from d , vertices b and e will be the new sources and enqueued into Q . Process continues. Finally, we assign each vertex a level as shown in Figure 4(a) and can move on to the next major step.

Positioning with Minimum Offset step will decide the final position of each vertex in the broadcast. The γ value of each vertex can be calculated

Input: A weighted DAG, $G = (V, E)$ with source s .

Output: A topological-ordering broadcast schedule with minimized latency.

```

/* Assigning level step */
(1)  $s$  is in the queue  $Q$  and assigned with level 0,
     $level(s) = 1$ ; each vertex  $u$  adjacent from  $s$ 
    is assigned with  $level(u) = 2$ ;  $V' = V - \{s\}$ ;
(2) while  $V'$  is not empty do
    (2.1) Pop vertex  $s$  from  $Q$ ;
    (2.2) for each vertex  $u$  adjacent from  $s$  do
        (2.2.1) if  $level(s) + 1 > level(u)$  then
             $u$  is assigned with the level
             $level(u) = level(s) + 1$ ;
        end if
        (2.2.2) Decrease the in-degree of  $u$  by one;
        (2.2.3) if the in-degree of  $u$  is zero then
            Insert  $u$  into  $Q$  and delete it from
             $V'$ ;
        end if
        (2.2.4) Derive the  $\gamma(u)$  by
             $\gamma(u) = \gamma(s) + \gamma(s) \cdot p_{su}$ ;
    end for
end while
/* Positioning with minimum offset step */
(3) for each level  $l$  do
    /* (from level 1 to higher level) */
    Vertex with greater  $\gamma$  value is placed earlier in
    the schedule (breaking tie by randomly selecting
    a vertex);
end for
    
```

Figure 2: Forward Minimum Offset Algorithm.

during the Assigning Level step. The position depends first on the level. If two or more vertices have the same level, we then consider the γ value. For example, in Figure 4(a), vertices c and d are on the same level. Since $\gamma(c) > \gamma(d)$, vertex c should be placed at an earlier position than vertex d . The resulting broadcast schedule for the DAG in Figure 3 is shown in Figure 4(b).

5.2 Time Complexity

It is not difficult to derive the time complexity for FMOA. For FMOA, the number of times to traverse a vertex v depends on the in-degree of v and the running time spent on each vertex is constant. If the maximum in-degree among all the vertices of the input DAG is d_i , the time complexity of FMOA is $O(d_i n)$ where n is the number of vertices in the DAG.

5.3 The Correctness of FMOA

In this section, we will show that FMOA can produce a broadcast which is a topological ordering of the vertices in the input DAG. To argue this, we first claim the following property about DAG.

Theorem 2 *Given a DAG $G = (V, E)$, suppose $v \in V$ and $v_1, v_2, \dots, v_l$ are the vertices adjacent from v . Then, there is at least one vertex v_i having no in-edge which is from either v_j or all the vertices reachable from v_j for all $j \neq i$.* $\square$

proof: Assume that for all v_i , there is an in-edge which is from either v_j or all the vertices reachable from v_j where $j \neq i$. W.L.O.G, suppose v_i has a in-edge which is from either v_{i+1} or all the vertices reachable from v_{i+1} for $i = 1, \dots, l-1$ and v_l has a in-edge which is from either v_k or all the vertices reachable from v_k where $k \in \{1, 2, \dots, l-1\}$. Then there is a loop in this graph which contradicts to the definition of DAG. $\square$

We use the above theorem to show that the broadcast schedule generated by FMOA is a topological ordering of the vertices. Recall FMOA, the traversed edges will be removed and never traversed again and a vertex becomes a source after all of its in-edges are removed. Whenever a source v has been traversed, by Theorem 2, at least one new source u will be produced. By keeping the topological ordering among vertices, the level of u is the level of v plus 1. Since all the vertices will be the source only once, such a rule therefore will guarantee the order of two adjacent vertices in the broadcast schedule. Thus, the generated broadcast schedule is a topological ordering of the vertices. We therefore can conclude the following theorem.

Theorem 3 *The broadcast schedule generated by FMOA is a topological ordering of the vertices of the input DAG.* $\square$

5.4 Bounds on the Latency

We now discuss the performance of FMOA. Consider an n -vertex weighted DAG $G = (V, E)$. Recall that, for each $v \in V$,

$$\gamma(v) = \sum_{P(s,v) \in \mathcal{P}_v} \prod_{(i,j) \in P(s,v)} p_{ij}.$$

Let γ_{min} denote $\min_{v \in V} \{\gamma(v)\}$. Suppose that f^* is the best broadcast schedule and f is the generated broadcast schedule, respectively. We first

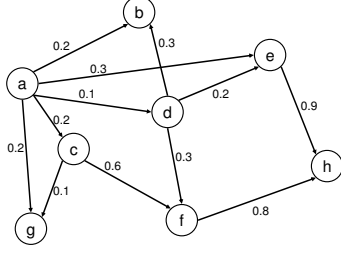
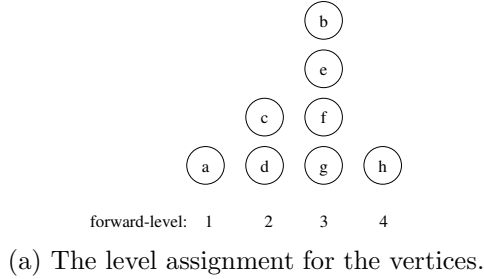


Figure 3: A weighted DAG where vertex a is a source and b, g , and h are the sinks.



vertex i	a	b	c	d	e	f	g	h
$\gamma(i)$	1	0.23	0.2	0.1	0.32	0.15	0.22	0.408
position	1	5	2	3	4	7	6	8

(b) The corresponding γ value and the resulting position for each vertex in the broadcast.

Figure 4: Illustration of algorithm FMOA with the DAG in Figure 3.

give a lower bound (LB) of the latency for the TOBML problem in Lemma 1 and then derive the upper bound (UB) of the latency for the broadcast schedule generated by FMOA in Lemma 2. Last, we compare these bounds with the latency of the best broadcast schedule in asymptotic approach.

Lemma 1 *The latency for the TOBML problem has a lower bound (LB) as*

$$\frac{n(n+1)}{2} \cdot \gamma_{\min}. \quad (3)$$

□

proof: W.L.O.G., we can rename all the vertices such that $f^*(v_i) = i$ for $i = 1, \dots, n$. Then the best average latency for accessing all the vertices is

$$\begin{aligned} \sum_{v_i \in V} f^*(v_i) \cdot \gamma(v_i) &= \sum_{v_i \in V} i \cdot \gamma(v_i) \geq \frac{n(n+1)}{2} \cdot \gamma_{\min} \\ &= LB. \end{aligned}$$

□

Lemma 2 *If k is the largest level number, then the upper bound of the latency for the broadcast schedule generated by FMOA is $k \cdot n$.*

proof: Let the vertices assigned to the same level number be a group. Since the largest level number is k , we can form k groups by these k levels. We then number the groups by the level numbers and suppose that each group has m_i vertices for $i = 1, \dots, k$. Let p_i be the maximum γ value among all the vertices in group i for $i = 1, \dots, k$. Please note that $p_i \leq 1$ for $i = 1, \dots, k$ since, for any vertex v in V , $\gamma(v) \leq 1$. Then the average latency for the broadcast schedule generated by FMOA is

$$\begin{aligned} \sum_{v \in V} f(v) \cdot \gamma(v) &\leq p_1 \cdot |m_1| + p_2 \cdot (|m_1| + |m_2|) \\ &\quad + \dots \\ &\quad + p_k \cdot (|m_1| + |m_2| + \dots + |m_k|) \\ &\leq n \cdot (p_1 + p_2 + \dots + p_k) \\ &\leq k \cdot n = UB \end{aligned}$$

□

Having the lower and upper bounds derived in Lemma 1 and 2, we can derive the asymptotic bound for the ratio, $\rho(n)$, of the latency for the broadcast schedule generated by FMOA to the latency of the best broadcast schedule. The bound of the ratio $\rho(n)$ indicates the performance when applying FMOA. From Lemma 1 and 2, we can derive

$$LB \leq \sum_{v_i \in V} f^*(v_i) \cdot \gamma(v_i) \leq \sum_{v_i \in V} f(v_i) \cdot \gamma(v_i) \leq UB. \quad (4)$$

Thus,

$$\begin{aligned} \rho(n) = \frac{LB}{UB} &= \frac{\frac{n(n+1)}{2} \cdot \gamma_{\min}}{kn} \\ &= \frac{n+1}{2k} \cdot \gamma_{\min}. \end{aligned} \quad (5)$$

Let ε be $\frac{\gamma_{\min}}{2k}$. We can conclude the following theorem.

Theorem 4 *The latency of the broadcast schedule generated by FMOA is with a factor of $\rho(n) = O(n\varepsilon)$ of the latency of a best broadcast schedule, where $\varepsilon = \frac{\gamma_{\min}}{2k}$.*

6 Greedy Traversal Heuristic

In this section, we provide a algorithm based on the greedy traversal on the DAG to generate the broadcast schedules where the topological ordering among the vertices is kept and the overall latency is minimized. The algorithm, Prior-V, generates the broadcast by considering the γ value of each vertex during the traversal. As for algorithms Prior-V, use different scheme to traverse the input DAG and the resulting traversal is not necessarily a depth-first traversal.

6.1 Prior-V Algorithm

In this subsection, we present algorithm, Prior-V, which considers the maximum probability of the prior vertices (*i.e.* the γ value) for each vertex during the traversal and then show the resulting broadcast is a topological ordering of the input DAG. Recall that we denote the position of a vertex v in the broadcast as $f(v)$. Algorithm Prior-V starts from the source s . Due to the topological ordering, the source must be assigned to position 1, *i.e.* $f(s) = 1$.

By deleting s and all the edges incident from it, we can have at least one new source. The next vertex to be visited is the one having the maximum γ value among all the new sources. Suppose vertex v is the next vertex to be visited. Then, we assign vertex v to position 2 (*i.e.*, $f(v) = 2$). The algorithm then again removes v and all the edges incident from v . Again, there is at least one new source. With the set of sources, the same process continues until all the vertices are traversed.

Figure 5 shows a high-level description of algorithm Prior-V. We use a maximum heap H to manage the new sources with the γ value as the key. When traversing a vertex v , v is assigned

Input: A weighted DAG, $G = (V, E)$ with source s .

Output: A topological ordering broadcast schedule with minimized latency.

```

(1) insert the  $s$  into  $H$ ; /*  $H$  is a maximum heap with
     $\gamma$  value as the key */
(2)  $pos = 1$  /*  $pos$ : the current position */
(3) while  $H \neq \text{empty}$ 
    (a) extract one vertex with the largest  $\gamma$  value
        from  $H$ ;
    (b) assign the extracted vertices to position  $pos$ ;
    (c) remove the extracted vertices and all the edges
        incident from them from the  $G$ ;
    (d) if there are new sources after (d) then
        insert the new sources into  $H$ ;
    (e)  $pos = pos + 1$ ;
end while
    
```

Figure 5: Prior-V Algorithm.

to the position currently considered and the new sources are inserted into H after vertex v and all the edges incident from v have been deleted. Then, we consider the next position. The next vertex to be visited can be extracted from H which has the maximum γ value in H . The process then repeats until H is empty.

Consider the DAG in Figure 3. The source a is assigned to the first position. After deleting a from the DAG, we insert new sources c and d into the maximum heap H . Then algorithm Prior-V extracts c and assigns it to position 2 since $\gamma(c) > \gamma(d)$. After deleting c and all the edges incident from c , the new source g is inserted into H . Again, algorithm Prior-V extracts the source having the largest γ value in H (g in this example) and assigns it to position 3. The process repeats until H is empty and the resulting broadcast schedule is $acgdebfbh$.

6.2 Correctness

We now show that the broadcast schedule generated by algorithm Prior-V is a topological ordering of the vertices for the input DAG. Consider the heap H which we use in algorithm Prior-V. Suppose algorithm Prior-V is about to extract a source from H to assign some position. The prior vertices of all the sources in the current H have been assigned and there is no topological ordering relation among the sources in H . Hence, for the sources extracted from H , all their prior vertices are assigned to earlier positions. We therefore can conclude the following theorem.

Theorem 5 *The broadcast schedule generated by*

algorithm Prior-V is a topological ordering of the vertices of the input DAG.

6.3 Time Complexity

In this section, we discuss the time complexity of algorithm Prior-V. Each operation used in the algorithm, like insertion and deletion, on the maximum heap H can be done in $O(\log n)$ time where n is the number of vertices in the input DAG. The operation of assigning vertices to channel needs only constant time. Since the heap H contains at most n number of vertices, the time complexity of algorithm Prior-V is $O(n \log n)$.

6.4 Bounds

Recall that γ_{min} denote $\min_{u \in V} \{\gamma(u)\}$ for a given DAG $G = (V, E)$. Similarly, we let γ_{max} denote $\max_{u \in V} \{\gamma(u)\}$. As shown in Lemma 1, a lower bound on the latency for the TOBML problem is

$$\frac{n(n+1)}{2} \cdot \gamma_{min}. \quad (6)$$

We now present the upper bound on the latency of the broadcast schedule generated by algorithm Prior-V. We start with the following lemma which will be used to derive the upper bound.

Lemma 3 *Given positive integers $a_1, a_2, \dots, a_k$. If $a_1 + a_2 + \dots + a_k = M$, then the following inequality holds,*

$$\prod_{i=1}^k a_i \geq M - k \quad (7)$$

where $1 \leq k \leq M$.

proof: We will show this by the induction on k . When $k = 1$, obviously $a_1 = M \geq M - 1$. Assume Inequality (7) holds for $k = n$. Then, when $k = n + 1$,

$$\begin{aligned}
 \prod_{i=1}^{n+1} a_i &= a_{n+1} \prod_{i=1}^n a_i \geq a_{n+1} \times (M - n) \\
 &\geq 1 \cdot (M - n) \quad (\text{since } a_{n+1} \text{ is a positive integer}) \\
 &\geq M - (n + 1).
 \end{aligned}$$

□

To derive an upper bound on the latency, we consider the possibly worst position for a vertex to be assigned. For a vertex v , we define $bl(v)$ be the number of vertices on the longest path among all the paths from v to sink(s). Note that, for a vertex v , $n - (bl(v) - 1)$ implies the worst possible position for v in the broadcast schedule.

Theorem 6 *The upper bound on the latency for the broadcast schedule generated by Prior-V is*

$$\gamma_{max} \cdot \left\{ n^2 + \frac{3}{2}n - k(k!(n-k))^{\frac{1}{k}} - \frac{k}{2}(n-k)^{\frac{2}{k}} \right\},$$

where n is the number of vertices in the input DAG and k the number of levels.

proof: Let $f(v)$ be the position of vertex v in the broadcast generated by one of the two algorithms. In order to keep the topological ordering of the vertices in the schedule, the possibly worst position for a vertex v to be assigned is $n - (bl(v) - 1)$. Since there are k levels, $bl(v)$ is between 1 to k for a vertex v . We let b_i denote the set of all the vertices having the same worst position $n - (i - 1)$, $i = 1, \dots, k$. $\{b_1, \dots, b_k\}$ is therefore a partition of the vertex set V of G .

We first consider the total latency of all the vertices in each b_i . For each b_i , $i = 1, \dots, k$, we have

$$\begin{aligned} \sum_{u \in b_i} f(u) \cdot \gamma(u) &\leq \gamma_{max} \cdot \sum_{u \in b_i} f(u) \\ &\leq \gamma_{max} \cdot \{(n - (i - 1)) + (n - (i - 1) - 1) + \dots \\ &\quad + (n - (i - 1) - (|b_i| - 1))\} \\ &\leq \gamma_{max} \cdot \{n|b_i| - (i - 1)|b_i| - \frac{1}{2}|b_i|^2 + \frac{1}{2}|b_i|\}. \end{aligned} \quad (8)$$

Then the total latency of all the vertices is:

$$\begin{aligned} &\sum_{i=1}^k \sum_{u \in b_i} f(u) \cdot \gamma(u) \\ &\leq \gamma_{max} \cdot \sum_{i=1}^k \{n|b_i| - (i - 1)|b_i| - \frac{1}{2}|b_i|^2 + \frac{1}{2}|b_i|\} \\ &= \gamma_{max} \cdot \left\{ n^2 + \frac{3}{2}n - \sum_{i=1}^k i|b_i| - \frac{1}{2} \sum_{i=1}^k |b_i|^2 \right\}. \end{aligned} \quad (9)$$

We now consider each term in Equation (9) using the Cauchy's Formula and Lemma 3 and can get the following two inequalities:

$$\sum_{i=1}^k i|b_i| \geq k \cdot (k!|b_1| \cdot |b_2| \cdots |b_k|)^{\frac{1}{k}} \geq k(k! \cdot (n - k))^{\frac{1}{k}} \quad (10)$$

and

$$\frac{1}{2} \sum_{i=1}^k |b_i|^2 \geq \frac{1}{2} k (|b_1| \cdot |b_2| \cdots |b_k|)^{\frac{2}{k}} \geq \frac{k}{2} (n - k)^{\frac{2}{k}}. \quad (11)$$

By applying (10) and (11) to (9), we can conclude the proof. $\square$

7 Simulation

This section presents the simulation results of the proposed algorithms. We measure the total latency, $\sum_{v \in V} f(v) \cdot \gamma(v)$. All the DAGs in the experiment are generated by the BNGenerator [11]

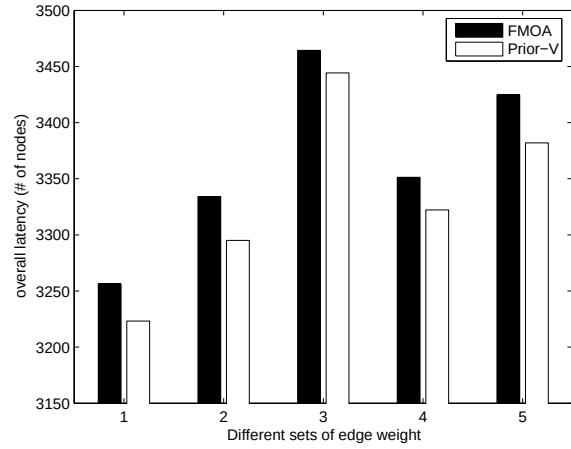


Figure 6: Latency comparison for the FMOA and Prior-V on a DAG using 5 difference sets of edge weight.

which can generates a DAG randomly. In our experiments, each DAG is of size 1,000 and the degree of each vertex in the DAG ranges from one to seven. The weight on each edge in a DAG is also generated randomly under the constraint that the summation of all the weights from a vertex is less than or equal to one. For each DAG, we run 1,000 different sets of edge weight and each set has a different ε value.

Figure 6 shows the latency when applying FMOA and Prior-V on a DAG with five different sets of edge weight. The Prior-V always generates a schedule with a shorter latency than the others. Recall that Prior-V assigns the vertex having larger γ value to an earlier position in each step and the γ value of a vertex indicates the probability to refer that vertex. Placing the vertex with larger γ value to an earlier position leads to a shorter overall latency than considering the level first due to the limitation on the position posed by the levels.

8 Conclusions

In this paper, we discuss how to generate the broadcast schedule for a given data set having DAG access pattern in wireless mobile environments. The generated broadcast schedule is a topological ordering of the vertices of the DAG and minimizes the overall latency. We formulate such a problem as the TOBML problem and show

that the TOBML problem is NP-complete. Due to the intractability, two different heuristics are presented: FMOA and Prior-V. We further analyze the proposed algorithms on the complexity and latency by giving the lower and upper bounds on the latency of the broadcast schedule. Last, we present the simulation results of these heuristics and compare the performance in terms of the latency.

References

- [1] S. Acharya, M. Franklin, and S. Zdonik. Balancing push and pull for data broadcasts. In *Proceedings of the 1997 ACM SIGMOD International Conference on Management of Data*, pages 183–194, May 1997.
- [2] A. Bar-Noy, J. Naor, and B. Schieber. Pushing dependent data in clients-providers-servers systems. *Wireless Networks*, 9:421–430, 2003.
- [3] G. Chartrand and O. R. Oellerman. *Applied and Algorithmic Graph Theory*. McGraw-Hill, 1993.
- [4] Y.C. Chehadeh, A.R. Hurson, and M. Kavehrad. Object organization on a single broadcast channel in the mobile computing environment. *Multimedia Tools and Applications*, 9:69–94, 1999.
- [5] S. Even and Y. Shiloah. Np-completeness of several arrangement problems. Technical report, Department of Computer Science, Israel Institute of Technology, Haifa, Isreal, 1975.
- [6] S. Hambrusch, C.-M. Liu, W.G. Aref, and S. Prabhakar. Query processing in broadcasted spatial index trees. *Lecture Notes in Computer Science*, 2121:502–510, 2001.
- [7] S. Hameed and N. Vaidya. Efficient algorithms for scheduling data broadcast. *ACM/Baltzer Journal of Wireless Networks*, 5(3):183–193, 1999.
- [8] J.-L. Huang, M.-S. Chen, and W.-C. Peng. Broadcasting dependent data for ordered queries without replication in a multi-channel mobile environment. In *Proceedings of the 19th International Conference on Data Engineering*, pages 692–694, 2003.
- [9] H.-P. Hung and M.-S. Chen. On exploring channel allocation in the diverse data broadcasting environment. In *ICDCS '05: Proceedings of the 25th IEEE International Conference on Distributed Computing Systems (ICDCS'05)*, pages 729–738, 2005.
- [10] A.R. Hurson and Y. Jiao. Data broadcasting in a mobile environment. In *Wireless Information Highway*, chapter 4, pages 96–154. IRM Press, 2004.
- [11] J. S. Ide, F. G. Cozman, and F. T. Ramos. Generating random bayesian networks with constraints on induced width. In *Proceedings of the 16th European Conference on Artificial Intelligence*, pages 323–327, 2004.
- [12] T. Imieliński, S. Viswanathan, and B. R. Badrinath. Data on air: Organization and access. *IEEE Transactions on Knowledge and Data Engineering*, 9(3):353–372, May/June 1997.
- [13] G. Lee, S.-C. Lo, and A.L.P. Chen. Data allocation on wireless broadcast channels for efficient query processing. *IEEE Transactions on Computers*, 51(10), 2002.
- [14] Yaw-Ling Lin. Minimum back-walk-free latency problem. In *COCOON*, pages 525–534, 2002.
- [15] C.-M. Liu. *Broadcasting and blocking large data sets with an index tree*. PhD thesis, Purdue University, West Lafayette, IN, 2002.
- [16] C.-M. Liu, L.-C. Wang, L. Chen, and C.-J. Chang. On-demand data disseminating with considering channel interference for efficient shortest-route service on intelligent transportation system. In *Proceedings of the 2004 IEEE International Conference on Networking, Sensing and Control*, pages 701–706, 2004.
- [17] B. Zheng, X. Wu, X. Jin, and D. L. Lee. Tosa: a near-optimal scheduling algorithm for multi-channel data broadcast. In *MEM '05: Proceedings of the 6th international conference on Mobile data management*, pages 29–37, 2005.